# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

**Department of Artificial Intelligence and Data Science**

## ASSESSMENT TOOL 1 – ASSIGNMENT 1

| | |
|---|---|
| Course Code: DSA03 | Course Title: Natural Language Processing |
| CO Assessed: CO1 – Imitation (L1) | Assessment: Assessment Tool 1 |
| Weightage: 15% of CIA | Questions: 10 (10 marks each) |

**CO5 Statement:** Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-2 & 3)

**Student Name:** V.NAVEEN **Reg. No.:** 192424126

**Section / Batch:** SLOT-B/AIDS **Date:** 6-11-2026

### Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

### Evaluation (for staff use)

| Q. No. | Understanding of the NLP concept (2) | Python implementation and logic (3) | Correctness of output (2) | Analysis and interpretation of results (2) | Code readability and documentation (1) | TOTAL |
|---|---|---|---|---|---|---|
| 1 | | | | | | |
| 2 | | | | | | |
| 3 | | | | | | |
| 4 | | | | | | |
| 5 | | | | | | |
| 6 | | | | | | |
| 7 | | | | | | |
| 8 | | | | | | |
| 9 | | | | | | |
| 10 | | | | | | |
| | | | | | **TOTAL MARKS** | |

# Question 1: Regular Expression-Based Text Processing

*BTL 3 – Apply | 10 Marks*

## Problem Statement

Write a Python program that uses regular expressions on a paragraph to extract email addresses and phone numbers, words that start with a capital letter, numerical values, and words that end in ing or ed. The results have to be displayed separately.

## Aim and Concept

A regular expression (regex) is a short pattern that describes a family of strings. Instead of checking a text one character at a time, I describe what the thing I am looking for looks like, for example some word characters, an @ symbol, a domain name, a dot and an ending, and Python's re module finds every place where that pattern fits.

In this program I wrote one pattern for each language feature in the question and ran the patterns over a short paragraph with re.findall(). Each group of results is printed on its own with a count.

## Approach

1. Store the paragraph, which contains emails, phone numbers, a date and some counts, in a string.
2. Write a pattern for email addresses (name, @, domain with one or more dots) and one for phone numbers (an optional +91 followed by 5 + 5 digits, or an area code followed by 8 digits).
3. Remove the emails and phone numbers from a copy of the text, so their digits and letters are not picked up again by the later patterns.
4. Write patterns for capitalised words, for numbers (whole numbers and decimals), and for words ending in ing or ed. The word must have at least three letters before the ending, so short words like red and bed are skipped.
5. Print every result list separately, with the number of matches found.

## Python Program

```
# Question 1: Regular expression based text processing
import re


text = """Dr. Priya Raman joined Saveetha Engineering College in Chennai on 12 June 2021.
She was teaching Natural Language Processing to 62 students and handled 3 labs every week.
For queries, students contacted her at priya.raman@saveetha.com or at nlp_lab@simats.edu.in.
The office also listed the numbers +91 98765 43210 and 044-2680 1234. Last semester the
department conducted 4 workshops, which attracted 250 participants and a budget of 1.5 lakh.
Many learners enjoyed the sessions and finished the assigned projects before the deadline."""


# 1. emails and phone numbers
email_pat = r'\b[\w.+-]+@[\w-]+(?:\.[\w-]+)+\b'
phone_pat = r'(?:\+91[-\s]?)?\b\d{5}[-\s]?\d{5}\b|\b0\d{2,4}[-\s]?\d{4}[-\s]?\d{4}\b'


emails = re.findall(email_pat, text)
phones = re.findall(phone_pat, text)


# remove emails and phones first so their digits are not counted as plain numbers later
clean = re.sub(phone_pat, ' ', re.sub(email_pat, ' ', text))
```

```
# 2. words that start with a capital letter
capital_words = re.findall(r'\b[A-Z][a-z]*\b', clean)


# 3. numerical values (integers and decimals)
numbers = re.findall(r'\b\d+(?:\.\d+)?\b', clean)


# 4. words ending with ing / ed (at least 3 letters before the ending)
ing_words = re.findall(r'\b[A-Za-z]{3,}ing\b', clean)
ed_words = re.findall(r'\b[A-Za-z]{3,}ed\b', clean)


def show(title, items):
    print(f"{title} ({len(items)} found)")
    for i, item in enumerate(items, 1):
        print(f"   {i:>2}. {item}")
    print()


print("=" * 60)
print("REGULAR EXPRESSION RESULTS")
print("=" * 60)
show("Email addresses", emails)
show("Phone numbers", [p.strip() for p in phones])
show("Words starting with a capital letter", capital_words)
show("Numerical values", numbers)
show("Words ending with 'ing'", ing_words)
show("Words ending with 'ed'", ed_words)
```

## Output

Terminal - python question_01.py

```
============================================================
REGULAR EXPRESSION RESULTS
============================================================
Email addresses (2 found)
    1. priya.raman@saveetha.com
    2. nlp_lab@simats.edu.in

Phone numbers (2 found)
    1. +91 98765 43210
    2. 044-2680 1234

Words starting with a capital letter (16 found)
    1. Dr
    2. Priya
    3. Raman
    4. Saveetha
    5. Engineering
    6. College
    7. Chennai
    8. June
    9. She
   10. Natural
   11. Language
   12. Processing
   13. For
   14. The
   15. Last
   16. Many
```

*Output 1.1 (part 1 of 2)*

```
Terminal  -  python question_01.py

Numerical values (7 found)
    1. 12
    2. 2021
    3. 62
    4. 3
    5. 4
    6. 250
    7. 1.5

Words ending with 'ing' (3 found)
    1. Engineering
    2. teaching
    3. Processing

Words ending with 'ed' (9 found)
    1. joined
    2. handled
    3. contacted
    4. listed
    5. conducted
    6. attracted
    7. enjoyed
    8. finished
    9. assigned
```

*Output 1.2 (part 2 of 2)*

## Analysis and Interpretation

The program found both email addresses and both phone numbers. The two phone numbers are written differently, +91 98765 43210 (mobile style) and 044-2680 1234 (landline with an area code), and one pattern with an OR in it handled both. Seven numbers were found: 12, 2021, 62, 3, 4, 250 and 1.5. The digits of the phone numbers are not in this list because I removed them first. Without that step, 98765 and 43210 would also have been reported as numbers.

Sixteen words begin with a capital letter, and this list shows that a capital is not the same thing as a name. Dr comes from the abbreviation Dr., and She, For, The, Last and Many are only capitalised because they start a sentence. To pick out real proper nouns we would need more than the first letter.

The ing list has three words, but only teaching is a verb form here, while Engineering and Processing are part of names. The ed list has nine words and all of them are past tense verbs. This shows the limit of the regex approach: it matches spelling only, so it cannot tell a verb ending in -ing from a noun ending in -ing. The patterns are also written for the formats that appear in this paragraph, and a number written as (044) 2680-1234 would need another pattern.

# Question 2: Finite State Automata for Word Pattern Recognition

*BTL 3 – Apply | 10 Marks*

## Problem Statement

Implement a finite state automaton in Python that checks whether a word starts with a vowel, ends with a consonant and contains consecutive vowels, and display whether each input word is accepted or rejected.

## Aim and Concept

A finite state automaton (FSA) is a very simple machine. It has a finite set of states, one start state, some accepting states, and a transition table that says which state to go to for each input symbol. The machine reads the word one symbol at a time, and when the word ends the word is accepted only if the machine is sitting in an accepting state.

Every letter is first turned into a class, V for a vowel (a, e, i, o, u) or C for a consonant (everything else, including y). That way each automaton has only two possible inputs. I built three small automata, one for each condition, and a word is accepted only when all three accept it.

## Approach

1. Convert each letter of the word to V or C.
2. Starts-with-vowel automaton: S0 goes to S1 (accepting) on V and to a dead state on C. S1 stays in S1 for any letter.
3. Ends-with-consonant automaton: two states, E0 and E1. Reading C always goes to E1 and reading V always goes back to E0, so being in E1 means the last letter was a consonant.
4. Consecutive-vowels automaton: D0 (last letter was not a vowel), D1 (one vowel just seen) and D2 (two vowels together were seen, accepting, and never left).
5. Run every word through the three machines, print what each one decided and then the overall result, ACCEPTED or REJECTED.

## Python Program

```
# Question 2: Finite State Automaton for word pattern recognition
# Letters are first mapped to a class: V (vowel) or C (consonant). The automaton
# reads one class at a time, so the transition tables stay small.


VOWELS = set("aeiou")


def letter_class(ch):
    return "V" if ch in VOWELS else "C"


class DFA:
    """A small deterministic finite automaton."""
    def __init__(self, name, start, accepting, delta):
        self.name, self.start = name, start
        self.accepting, self.delta = accepting, delta


    def run(self, word):
        state = self.start
        path = [state]
        for ch in word:
            state = self.delta[(state, letter_class(ch))]
            path.append(state)
        return state in self.accepting, path
```

```
# FSA 1: word must start with a vowel
starts_vowel = DFA("Starts with vowel", "S0", {"S1"}, {
    ("S0", "V"): "S1", ("S0", "C"): "DEAD",
    ("S1", "V"): "S1", ("S1", "C"): "S1",
    ("DEAD", "V"): "DEAD", ("DEAD", "C"): "DEAD"})


# FSA 2: word must end with a consonant (accepting state = last letter was C)
ends_consonant = DFA("Ends with consonant", "E0", {"E1"}, {
    ("E0", "V"): "E0", ("E0", "C"): "E1",
    ("E1", "V"): "E0", ("E1", "C"): "E1"})


# FSA 3: word must contain two vowels side by side
has_double_vowel = DFA("Has consecutive vowels", "D0", {"D2"}, {
    ("D0", "V"): "D1", ("D0", "C"): "D0",
    ("D1", "V"): "D2", ("D1", "C"): "D0",
    ("D2", "V"): "D2", ("D2", "C"): "D2"})


machines = [starts_vowel, ends_consonant, has_double_vowel]


def check(word):
    word = word.lower()
    results = [m.run(word)[0] for m in machines]
    return results, all(results)


print("Transition tables (state, input) -> next state")
for m in machines:
    print(f"\n{m.name}: start={m.start}, accepting={sorted(m.accepting)}")
    for (s, c), t in m.delta.items():
        if s != "DEAD":
            print(f"   ({s}, {c}) -> {t}")


words = ["audit", "education", "apple", "outside", "object", "eat",
         "book", "idea", "ouch", "author", "island", "sky", "orange", "airport"]


print("\n" + "-" * 72)
print(f"{'Word':<12}{'Vowel start':<14}{'Cons. end':<12}{'VV inside':<12}{'Result'}")
print("-" * 72)
yn = lambda b: "yes" if b else "no"
for w in words:
    (a, b, c), ok = check(w)
    print(f"{w:<12}{yn(a):<14}{yn(b):<12}{yn(c):<12}{'ACCEPTED' if ok else 'REJECTED'}")


print("\nState trace for two sample words")
for w in ["author", "apple"]:
    print(f"  {w}:")
    for m in machines:
        print(f"     {m.name:<24}", " -> ".join(m.run(w)[1]))
```

## Output

```
Terminal  -  python question_02.py

Transition tables (state, input) -> next state

Starts with vowel: start=S0, accepting=['S1']
   (S0, V) -> S1
   (S0, C) -> DEAD
   (S1, V) -> S1
   (S1, C) -> S1

Ends with consonant: start=E0, accepting=['E1']
   (E0, V) -> E0
   (E0, C) -> E1
   (E1, V) -> E0
   (E1, C) -> E1

Has consecutive vowels: start=D0, accepting=['D2']
   (D0, V) -> D1
   (D0, C) -> D0
   (D1, V) -> D2
   (D1, C) -> D0
   (D2, V) -> D2
   (D2, C) -> D2

------------------------------------------------------------------------
Word        Vowel start   Cons. end   VV inside   Result
------------------------------------------------------------------------
audit       yes           yes         yes         ACCEPTED
education   yes           yes         yes         ACCEPTED
apple       yes           no          no          REJECTED
outside     yes           no          yes         REJECTED
object      yes           yes         no          REJECTED
eat         yes           yes         yes         ACCEPTED
book        no            yes         yes         REJECTED
idea        yes           no          yes         REJECTED
ouch        yes           yes         yes         ACCEPTED
```

*Output 2.1 (part 1 of 2)*

```
Terminal  -  python question_02.py

author      yes           yes         yes         ACCEPTED
island      yes           yes         no          REJECTED
sky         no            yes         no          REJECTED
orange      yes           no          no          REJECTED
airport     yes           yes         yes         ACCEPTED

State trace for two sample words
  author:
     Starts with vowel        S0 -> S1 -> S1 -> S1 -> S1 -> S1 -> S1
     Ends with consonant      E0 -> E0 -> E0 -> E1 -> E1 -> E0 -> E1
     Has consecutive vowels   D0 -> D1 -> D2 -> D2 -> D2 -> D2 -> D2
  apple:
     Starts with vowel        S0 -> S1 -> S1 -> S1 -> S1 -> S1
     Ends with consonant      E0 -> E0 -> E1 -> E1 -> E1 -> E0
     Has consecutive vowels   D0 -> D1 -> D0 -> D0 -> D0 -> D1
```

*Output 2.2 (part 2 of 2)*

## Analysis and Interpretation

Six of the fourteen words were accepted: audit, education, eat, ouch, author and airport. Each of them starts with a vowel, ends with a consonant and has a vowel pair (au, io, ea, ou, au, ai). The other eight fail for different reasons. Outside passes the first and third tests but ends in e. Object and island fail only because they have no vowel pair, and idea fails only because it ends in a vowel. Book fails only on its first letter. Apple and orange fail two tests (they end in e and have no vowel pair), and sky fails on the first letter and on the vowel pair.

The state trace for author shows how the third automaton works. It goes D0, then D1 after reading a, then D2 after u, and from then on it stays in D2, so nothing later in the word can undo the match. For apple it reaches D1 after the first a, falls back to D0 on p, and ends in D1 after the last e, which is not an accepting state, so the word is rejected.

One simplification is that I treat y as a consonant, so in the program's view sky has no vowels at all. A fuller version would decide by the position of y. The main idea of the exercise still holds. An FSA only has a fixed number of states, so it can remember simple things such as "was the last letter a vowel?" but it cannot count without a limit.

# Question 3: Morphological Analysis of English Words

*BTL 4 – Analyze | 10 Marks*

## Problem Statement

Write a Python program that finds the root word and affixes of English words, separates prefixes and suffixes, shows the morphological structure, and analyses unhappiness, replaying, disagreement and careless. The components must be displayed in a structured format.

## Aim and Concept

Morphology is the study of how words are built. The smallest meaningful pieces of a word are called morphemes. A word usually has a root that carries the core meaning, and it can have prefixes in front of the root and suffixes after it. For example, unhappiness is made of un-, happy and -ness.

A program cannot simply chop letters off the ends of a word, because that would split words like red into r and ed. So my analyser only accepts a split when what is left is a known root word in a small dictionary. It also undoes common spelling changes, such as happi back to happy, before it checks the dictionary.

## Approach

1. Keep a dictionary of root words, a list of prefixes (un, re, dis, mis, pre, in) and a list of suffixes with their meaning.
2. For each word, try no prefix first and then each prefix, longest first.
3. For the remaining part, strip suffixes one at a time (longest first) and check what is left against the dictionary. Also try spelling variants, such as adding an e or changing a final i back to y.
4. When a root is found, record the prefix, the root and the suffixes in the order they were added.
5. If nothing matches, treat the whole word as its own root.
6. Print a summary table, then a detailed breakdown of the four required words.

## Python Program

```
# Question 3: Morphological analysis of English words
# Idea: peel off prefixes and suffixes, and accept the split only when what is
# left is a known root word. This stops us from cutting "red" into "r" + "ed".


ROOTS = {"happy", "play", "agree", "care", "kind", "heat", "understand",
         "teach", "possible", "appear", "visit", "fair", "work", "view",
         "write", "like", "help", "real"}


PREFIXES = {"un": "not / opposite", "re": "again", "dis": "not / opposite of",
            "mis": "wrongly", "pre": "before", "in": "not"}


# suffix -> (what it does)
SUFFIXES = {"ness": "noun-forming, state of", "ment": "noun-forming, result of",
            "less": "adjective-forming, without", "ful": "adjective-forming, full of",
            "ing": "inflectional, present participle", "ed": "inflectional, past tense",
            "ly": "adverb-forming, in the manner of", "er": "noun-forming, one who",
            "able": "adjective-forming, can be", "s": "inflectional, plural or 3rd person"}


def root_variants(stem):
    """Undo common spelling changes: happi -> happy, writ -> write, etc."""
    options = [stem, stem + "e"]
    if stem.endswith("i"):
```

```
        options.append(stem[:-1] + "y")
    if len(stem) > 2 and stem[-1] == stem[-2]:
        options.append(stem[:-1])
    return options


def strip_suffixes(word):
    """Return (root, [suffixes in the order they were added]) or None."""
    if word in ROOTS:
        return word, []
    for suf in sorted(SUFFIXES, key=len, reverse=True):
        if word.endswith(suf) and len(word) > len(suf) + 1:
            for cand in root_variants(word[:-len(suf)]):
                found = strip_suffixes(cand)
                if found:
                    return found[0], found[1] + [suf]
    return None


def analyse(word):
    for pre in [""] + sorted(PREFIXES, key=len, reverse=True):
        if word.startswith(pre):
            result = strip_suffixes(word[len(pre):])
            if result:
                return pre, result[0], result[1]
    return "", word, []          # nothing found: the word is its own root


def structure(pre, root, sufs):
    parts = ([f"{pre}-"] if pre else []) + [root] + [f"-{s}" for s in sufs]
    return " + ".join(parts)


words = ["unhappiness", "replaying", "disagreement", "careless",
         "unkindly", "preheated", "mislikes", "teacher", "visit"]


print("=" * 78)
print(f"{'Word':<14}{'Prefix':<9}{'Root':<12}{'Suffix(es)':<12}Structure")
print("=" * 78)
for w in words:
    pre, root, sufs = analyse(w)
    print(f"{w:<14}{pre or '-':<9}{root:<12}{', '.join(sufs) or '-':<12}"
          f"{structure(pre, root, sufs)}")


print("\nDetailed breakdown of the four required words")
print("-" * 78)
for w in ["unhappiness", "replaying", "disagreement", "careless"]:
    pre, root, sufs = analyse(w)
    print(f"{w}")
    print(f"   root     : {root}")
    if pre:
        print(f"   prefix   : {pre}-  ({PREFIXES[pre]})")
    for s in sufs:
        print(f"   suffix   : -{s}  ({SUFFIXES[s]})")
    print(f"   morphemes: {len(sufs) + (1 if pre else 0) + 1}")
    print()
```

## Output

Terminal - python question_03.py

```
================================================================================
Word          Prefix   Root        Suffix(es)  Structure
================================================================================
unhappiness   un       happy       ness        un- + happy + -ness
replaying     re       play        ing         re- + play + -ing
disagreement  dis      agree       ment        dis- + agree + -ment
careless      -        care        less        care + -less
unkindly      un       kind        ly          un- + kind + -ly
preheated     pre      heat        ed          pre- + heat + -ed
mislikes      mis      like        s           mis- + like + -s
teacher       -        teach       er          teach + -er
visit         -        visit       -           visit

Detailed breakdown of the four required words
--------------------------------------------------------------------------------
unhappiness
   root     : happy
   prefix   : un-  (not / opposite)
   suffix   : -ness  (noun-forming, state of)
   morphemes: 3

replaying
   root     : play
   prefix   : re-  (again)
   suffix   : -ing  (inflectional, present participle)
   morphemes: 3
```

*Output 3.1 (part 1 of 2)*

Terminal - python question_03.py

```
disagreement
   root     : agree
   prefix   : dis-  (not / opposite of)
   suffix   : -ment  (noun-forming, result of)
   morphemes: 3

careless
   root     : care
   suffix   : -less  (adjective-forming, without)
   morphemes: 2
```

*Output 3.2 (part 2 of 2)*

## Analysis and Interpretation

All four required words were split the way a dictionary would split them. Unhappiness is un- + happy + -ness, replaying is re- + play + -ing, disagreement is dis- + agree + -ment, and careless is care + -less. The first three have three morphemes each, and careless has two because it has no prefix.

Two things are worth noticing. First, unhappiness needed the spelling rule. The word contains happi, but the root is happy, and without the i to y rule the dictionary lookup would fail. Second, the output lists the pieces in a flat row, but they actually join in layers. In unhappiness the prefix un- first attaches to the adjective happy to give unhappy, and then -ness turns that adjective into a noun, so the structure

is [[un- + happy] + -ness]. In the same way replaying is [[re- + play] + -ing] and disagreement is [[dis- + agree] + -ment].

The suffixes are also of different kinds. The suffixes -ness, -ment and -less are derivational because they make a new word (a noun or an adjective), while -ing is inflectional because it only gives a grammatical form of play. The extra words in the table follow the same pattern: unkindly, preheated and mislikes were split correctly, and visit has no affixes, so it is printed as one morpheme. The method depends on the root dictionary, so a word whose root is missing from the list will not be split.

# Question 4: Inflectional and Derivational Morphology

*BTL 3 – Apply | 10 Marks*

## Problem Statement

Write a Python program that finds plural nouns, past tense and present participle verbs, and words formed with prefixes and suffixes in a given text, classifies them as inflectional or derivational, and shows each original word with its classification.

## Aim and Concept

Inflectional morphology changes the form of a word to show grammar, such as plural (book, books), tense (walk, walked) or the -ing participle (walking). The word stays the same kind of word. Derivational morphology builds a new word, and it often changes the word class or the meaning, for example happy to unhappy, teach to teacher or quiet to quietly.

One word can have both kinds. In teachers the suffix -er is derivational (teach to teacher) and the -s is inflectional (plural). Derivational affixes sit closer to the root and inflectional ones go on the outside.

## Approach

1. Split the text into words and keep the first occurrence of each.
2. Check a small table of irregular forms first (children, flew), because these have no suffix to remove.
3. For the other words, remove prefixes and suffixes and check what is left against a small lexicon that also stores the part of speech (noun, verb, adjective). The part of speech decides what a suffix means: -s after a noun is a plural and -s after a verb is third person singular.
4. Mark each affix as inflectional (-s, -es, -ed, -ing) or derivational (prefixes, -er, -ly, -ful, -less, -ness).
5. Classify the word as Inflectional, Derivational or Both, and print the results together with a summary.

## Python Program

```
# Question 4: Inflectional and derivational morphology
import re
import textwrap


text = ("The teachers replayed the lessons while the students were studying quietly. "
        "Unhappy children copied their notes carefully, and the writers discussed "
        "several useful ideas. Kindness helps the learners, and a careless reader "
        "misses the point. The birds flew over the playground, singing loudly.")


# small lexicon of root words with their part of speech
LEX = {"teach": "V", "play": "V", "lesson": "N", "student": "N", "study": "V",
       "quiet": "A", "happy": "A", "child": "N", "copy": "V", "note": "N",
       "care": "N", "write": "V", "discuss": "V", "use": "N", "idea": "N",
       "kind": "A", "help": "V", "learn": "V", "read": "V", "miss": "V",
       "point": "N", "bird": "N", "playground": "N", "sing": "V", "loud": "A",
       "fly": "V"}


IRREGULAR = {"children": ("child", "plural noun"), "flew": ("fly", "past tense")}
PREFIXES = ["un", "re", "dis", "mis", "pre"]
```

```
# suffix: (kind, POS it attaches to, label, POS after attaching)
SUFFIXES = {
    "ing":  ("inflectional", "V", "present participle", "V"),
    "ed":   ("inflectional", "V", "past tense", "V"),
    "es":   ("inflectional", "NV", "plural / 3rd person", "same"),
    "s":    ("inflectional", "NV", "plural / 3rd person", "same"),
    "ness": ("derivational", "A", "-ness (adj -> noun)", "N"),
    "ly":   ("derivational", "A", "-ly (adj -> adverb)", "A"),
    "ful":  ("derivational", "N", "-ful (noun -> adj)", "A"),
    "less": ("derivational", "N", "-less (noun -> adj)", "A"),
    "er":   ("derivational", "V", "-er (verb -> noun)", "N"),
}


def variants(stem):
    out = [stem, stem + "e"]
    if stem.endswith("i"):
        out.append(stem[:-1] + "y")
    return out


def strip(word):
    """Return (root, root_pos, [(suffix, info)...], current_pos) or None."""
    if word in LEX:
        return word, LEX[word], [], LEX[word]
    for suf in sorted(SUFFIXES, key=len, reverse=True):
        kind, takes, label, gives = SUFFIXES[suf]
        if suf == "es" and not word[:-2].endswith(("s", "x", "z", "ch", "sh")):
            continue                    # "notes" is note + s, not not + es
        if word.endswith(suf) and len(word) > len(suf) + 1:
            for cand in variants(word[:-len(suf)]):
                inner = strip(cand)
                if inner and inner[3] in takes:
                    root, rpos, sufs, cur = inner
                    new_pos = cur if gives == "same" else gives
                    if suf in ("s", "es") and cur == "V":
                        info = (suf, "inflectional", "3rd person singular verb")
                    elif suf in ("s", "es"):
                        info = (suf, "inflectional", "plural noun")
                    else:
                        info = (suf, kind, label)
                    return root, rpos, sufs + [info], new_pos
    return None


def analyse(word):
    w = word.lower()
    if w in IRREGULAR:
        root, label = IRREGULAR[w]
        return {"root": root, "prefix": "", "sufs": [("(irregular)", "inflectional", label)]}
    for pre in [""] + PREFIXES:
        if w.startswith(pre):
            res = strip(w[len(pre):])
            if res and (pre or res[2]):
                return {"root": res[0], "prefix": pre, "sufs": res[2]}
    return None


def classify(info):
    infl = [s for s in info["sufs"] if s[1] == "inflectional"]
    deriv = [s for s in info["sufs"] if s[1] == "derivational"]
    has_deriv = bool(info["prefix"]) or bool(deriv)
    if infl and has_deriv:
        return "Both"
    return "Derivational" if has_deriv else "Inflectional"


words = re.findall(r"[A-Za-z]+", text)
rows, seen = [], set()
for w in words:
    if w.lower() in seen:
        continue
    seen.add(w.lower())
    info = analyse(w)
    if info:
        rows.append((w, info))


print(f"{'Word':<12}{'Morphemes':<22}{'Inflectional feature':<26}{'Class'}")
print("-" * 76)
for w, info in rows:
    parts = ([info["prefix"] + "-"] if info["prefix"] else []) + [info["root"]] \
          + ["-" + s[0] for s in info["sufs"] if s[0] != "(irregular)"]
    infl = ", ".join(s[2] for s in info["sufs"] if s[1] == "inflectional") or "-"
    print(f"{w:<12}{' + '.join(parts):<22}{infl:<26}{classify(info)}")


groups = {"Inflectional": [], "Derivational": [], "Both": []}
for w, info in rows:
    groups[classify(info)].append(w)
```

```
print("\nSummary")
print("-" * 76)
for g, ws in groups.items():
    line = f"{g} ({len(ws)}): " + ", ".join(ws)
    print(textwrap.fill(line, 76, subsequent_indent="    "))


print("\nSpecific forms found")
plural = [w for w, i in rows if any(s[2] == "plural noun" for s in i["sufs"])]
past = [w for w, i in rows if any(s[2] == "past tense" for s in i["sufs"])]
ing = [w for w, i in rows if any(s[2] == "present participle" for s in i["sufs"])]
for label, items in [("plural nouns", plural), ("past tense verbs", past),
                     ("present participles", ing)]:
    print(textwrap.fill(f"{label}: " + ", ".join(items), 76,
                        initial_indent="  ", subsequent_indent="      "))
```

## Output

Terminal - python question_04.py

| Word | Morphemes | Inflectional feature | Class |
|---|---|---|---|
| teachers | teach + -er + -s | plural noun | Both |
| replayed | re- + play + -ed | past tense | Both |
| lessons | lesson + -s | plural noun | Inflectional |
| students | student + -s | plural noun | Inflectional |
| studying | study + -ing | present participle | Inflectional |
| quietly | quiet + -ly | - | Derivational |
| Unhappy | un- + happy | - | Derivational |
| children | child | plural noun | Inflectional |
| copied | copy + -ed | past tense | Inflectional |
| notes | note + -s | plural noun | Inflectional |
| carefully | care + -ful + -ly | - | Derivational |
| writers | write + -er + -s | plural noun | Both |
| discussed | discuss + -ed | past tense | Inflectional |
| useful | use + -ful | - | Derivational |
| ideas | idea + -s | plural noun | Inflectional |
| Kindness | kind + -ness | - | Derivational |
| helps | help + -s | 3rd person singular verb | Inflectional |
| learners | learn + -er + -s | plural noun | Both |
| careless | care + -less | - | Derivational |
| reader | read + -er | - | Derivational |
| misses | miss + -es | 3rd person singular verb | Inflectional |
| birds | bird + -s | plural noun | Inflectional |
| flew | fly | past tense | Inflectional |
| singing | sing + -ing | present participle | Inflectional |
| loudly | loud + -ly | - | Derivational |

*Output 4.1 (part 1 of 2)*

```
Terminal  -  python question_04.py

Summary
--------------------------------------------------------------------------
Inflectional (13): lessons, students, studying, children, copied, notes,
    discussed, ideas, helps, misses, birds, flew, singing
Derivational (8): quietly, Unhappy, carefully, useful, Kindness, careless,
    reader, loudly
Both (4): teachers, replayed, writers, learners

Specific forms found
  plural nouns: teachers, lessons, students, children, notes, writers,
      ideas, learners, birds
  past tense verbs: replayed, copied, discussed, flew
  present participles: studying, singing
```

*Output 4.2 (part 2 of 2)*

## Analysis and Interpretation

The paragraph gave 25 analysable words: 13 are purely inflectional, 8 purely derivational and 4 have both. The plural nouns found were teachers, lessons, students, children, notes, writers, ideas, learners and birds. The past tense verbs were replayed, copied, discussed and flew, and the present participles were studying and singing. The words helps and misses are third person singular verbs, which are inflectional too.

The four words in the Both group (teachers, replayed, writers and learners) are the most interesting. Each has a derivational part next to the root and an inflectional part on the outside, for example re- + play + -ed. Words like quietly, carefully, useful, kindness, careless and unhappy are purely derivational because they change the word class or the meaning but carry no grammatical ending.

A few cases need care. Irregular forms such as children and flew cannot be found by removing letters, so they need the lookup table. The ending -s can be a plural or a verb ending, and the program decides which one from the part of speech of the root. Spelling changes are handled as well, for instance copied goes back to copy and notes goes back to note. Function words (the, while, were) and the compound playground are not listed because they are not built with affixes. The program only knows the words in its small lexicon, so a bigger text would need a bigger lexicon.

# Question 5: Porter Stemming Algorithm

*BTL 4 – Analyze | 10 Marks*

## Problem Statement

Implement the Porter Stemming Algorithm in Python, apply it to a paragraph, display the original and stemmed words, find the words that give the same stem, and analyse how stemming reduces different word forms to a common base. Example words: playing, played, studies, studying, connected, connection.

## Aim and Concept

Stemming cuts the endings off a word so that different forms of the same word end up as one string. For example playing, played and plays all become play. This is useful in search engines and text classification, where the different forms should count as the same term.

The Porter stemmer is built around the measure m of a word, which counts the vowel-consonant groups in it. For example tree has m = 0, play has m = 1 and connect has m = 2. A rule is applied only if the part that would be left is long enough, and this stops short words from being cut to nothing. The rules run in five phases, one after another. The stem does not have to be a real word. It only has to be the same for all the related forms.

## Approach

1. Step 1a handles plurals: sses becomes ss, ies becomes i, and a final s is removed.
2. Step 1b removes ed and ing when the remaining stem still has a vowel, then repairs the ending (at, bl and iz get an e, a double consonant is cut to one, and a short word gets an e).
3. Step 1c changes a final y to i when it comes after a consonant.
4. Steps 2 and 3 replace long derivational endings by shorter ones (ational to ate, ization to ize, ness to nothing) when m is above 0.
5. Step 4 removes endings such as al, er, ment, ion and ous when m is above 1.
6. Step 5 removes a final e and a double l when the word is long enough.
7. Run the stemmer on a paragraph, group the words by stem, and print the original and stemmed forms.

## Python Program

```
# Question 5: Porter stemming algorithm (written from scratch)
import re
from itertools import groupby
from collections import defaultdict


class PorterStemmer:
    def is_cons(self, w, i):
        c = w[i]
        if c in "aeiou":
            return False
        if c == "y":                    # y is a vowel after a consonant
            return i == 0 or not self.is_cons(w, i - 1)
        return True


    def measure(self, stem):
        """m in [C](VC)^m[V]: number of vowel-consonant sequences."""
```

```
    pattern = "".join("C" if self.is_cons(stem, i) else "V" for i in range(len(stem)))
    collapsed = "".join(k for k, _ in groupby(pattern))
    return collapsed.count("VC")


def has_vowel(self, stem):
    return any(not self.is_cons(stem, i) for i in range(len(stem)))


def double_cons(self, w):
    return len(w) > 1 and w[-1] == w[-2] and self.is_cons(w, len(w) - 1)


def cvc(self, w):
    return (len(w) > 2 and self.is_cons(w, len(w) - 3)
            and not self.is_cons(w, len(w) - 2) and self.is_cons(w, len(w) - 1)
            and w[-1] not in "wxy")


def replace(self, w, rules, min_m):
    """Apply the first rule whose suffix matches (if the measure allows it)."""
    for suf, rep in sorted(rules, key=lambda r: -len(r[0])):
        if w.endswith(suf):
            stem = w[:-len(suf)]
            return stem + rep if self.measure(stem) > min_m else w
    return w


def step1a(self, w):
    if w.endswith("sses"): return w[:-2]
    if w.endswith("ies"):  return w[:-2]
    if w.endswith("ss"):   return w
    if w.endswith("s"):    return w[:-1]
    return w


def step1b(self, w):
    if w.endswith("eed"):
        return w[:-1] if self.measure(w[:-3]) > 0 else w
    for suf in ("ed", "ing"):
        if w.endswith(suf) and self.has_vowel(w[:-len(suf)]):
            w = w[:-len(suf)]
            if w.endswith(("at", "bl", "iz")):
                return w + "e"
            if self.double_cons(w) and w[-1] not in "lsz":
                return w[:-1]
            if self.measure(w) == 1 and self.cvc(w):
                return w + "e"
            return w
    return w


def step1c(self, w):
    # y -> i when there is a vowel before it. Like NLTK, we also require the
    # letter before y to be a consonant, so "play" stays "play", not "plai".
    if w.endswith("y") and len(w) > 2 and self.is_cons(w, len(w) - 2):
        return w[:-1] + "i"
    return w


STEP2 = [("ational", "ate"), ("tional", "tion"), ("enci", "ence"), ("anci", "ance"),
         ("izer", "ize"), ("abli", "able"), ("alli", "al"), ("entli", "ent"),
         ("eli", "e"), ("ousli", "ous"), ("ization", "ize"), ("ation", "ate"),
         ("ator", "ate"), ("alism", "al"), ("iveness", "ive"), ("fulness", "ful"),
         ("ousness", "ous"), ("aliti", "al"), ("iviti", "ive"), ("biliti", "ble")]
STEP3 = [("icate", "ic"), ("ative", ""), ("alize", "al"), ("iciti", "ic"),
         ("ical", "ic"), ("ful", ""), ("ness", "")]
STEP4 = ["al", "ance", "ence", "er", "ic", "able", "ible", "ant", "ement", "ment",
         "ent", "ion", "ou", "ism", "ate", "iti", "ous", "ive", "ize"]


def step4(self, w):
    for suf in sorted(self.STEP4, key=len, reverse=True):
        if w.endswith(suf):
            stem = w[:-len(suf)]
            if suf == "ion" and not stem.endswith(("s", "t")):
                continue
            return stem if self.measure(stem) > 1 else w
    return w


def step5(self, w):
    if w.endswith("e"):
        m = self.measure(w[:-1])
        if m > 1 or (m == 1 and not self.cvc(w[:-1])):
            w = w[:-1]
    if self.measure(w) > 1 and w.endswith("ll"):
        w = w[:-1]
    return w
```

```
    def stem(self, word):
        w = word.lower()
        if len(w) <= 2:
            return w
        w = self.step1c(self.step1b(self.step1a(w)))
        w = self.replace(w, self.STEP2, 0)
        w = self.replace(w, self.STEP3, 0)
        w = self.step4(w)
        return self.step5(w)


ps = PorterStemmer()


# quick self-check against well known Porter outputs
known = {"caresses": "caress", "ponies": "poni", "cats": "cat", "agreed": "agre",
         "motoring": "motor", "hopping": "hop", "relational": "relat",
         "conditional": "condit", "hopefulness": "hope", "electrical": "electr",
         "adjustment": "adjust", "controll": "control", "happy": "happi"}
ok = sum(ps.stem(k) == v for k, v in known.items())
print(f"Self-check against standard Porter outputs: {ok}/{len(known)} correct\n")


paragraph = ("The students were studying hard. She played football and enjoys playing "
             "games with friends. They connected the devices, and the connection was "
             "strong. Her studies show that connected learners generate better results.")


words = re.findall(r"[a-z]+", paragraph.lower())
unique = list(dict.fromkeys(words))


print(f"{'Original':<14}{'Stem':<12}{'Chars removed'}")
print("-" * 40)
for w in unique:
    s = ps.stem(w)
    print(f"{w:<14}{s:<12}{len(w) - len(s)}")


print("\nRequired example words")
for w in ["playing", "played", "studies", "studying", "connected", "connection"]:
    print(f"   {w:<12}->  {ps.stem(w)}")


groups = defaultdict(list)
for w in unique:
    groups[ps.stem(w)].append(w)
print("\nWords that share the same stem")
for stem, ws in groups.items():
    if len(ws) > 1:
        print(f"   {stem:<10}<- {', '.join(ws)}")


before, after = sum(map(len, unique)), sum(len(ps.stem(w)) for w in unique)
print(f"\nCharacters before: {before}, after: {after} "
      f"({100 * (before - after) / before:.1f}% shorter)")


print("\nLimitations: different words reduced to one stem")
for w in ["generate", "general", "generous", "universe", "university"]:
    print(f"   {w:<12}->  {ps.stem(w)}")
```

## Output

Terminal - python question_05.py

```
Self-check against standard Porter outputs: 13/13 correct

Original      Stem        Chars removed
----------------------------------------
the           the         0
students      student     1
were          were        0
studying      studi       3
hard          hard        0
she           she         0
played        play        2
football      footbal     1
and           and         0
enjoys        enjoy       1
playing       play        3
games         game        1
with          with        0
friends       friend      1
they          they        0
connected     connect     2
devices       devic       2
connection    connect     3
was           wa          1
strong        strong      0
her           her         0
studies       studi       2
show          show        0
that          that        0
learners      learner     1
generate      gener       3
better        better      0
results       result      1
```

*Output 5.1 (part 1 of 2)*

```
Terminal  -  python question_05.py

Required example words
   playing     ->  play
   played      ->  play
   studies     ->  studi
   studying    ->  studi
   connected   ->  connect
   connection  ->  connect

Words that share the same stem
   studi     <- studying, studies
   play      <- played, playing
   connect   <- connected, connection

Characters before: 162, after: 134 (17.3% shorter)

Limitations: different words reduced to one stem
   generate    ->  gener
   general     ->  gener
   generous    ->  gener
   universe    ->  univers
   university  ->  univers
```

*Output 5.2 (part 2 of 2)*

## Analysis and Interpretation

Before using the stemmer on the paragraph I checked it on 13 words whose Porter stems are well known (caresses, ponies, agreed, motoring, relational, hopefulness and others), and all 13 matched. That gave me some confidence that the steps are implemented correctly.

The paragraph has 28 different words, and after stemming they have 134 characters instead of 162, which is 17.3% shorter. The six example words behave as expected. Playing and played both give play, studies and studying both give studi, and connected and connection both give connect. So six different word forms collapse into three stems, which is the common base the question asks about.

The stems are not always real words. Studi, devic and footbal are not English, but this does not matter because they are only labels used to match related forms. Stemming also makes mistakes, since it follows spelling and not meaning. The word was became wa because step 1a removed the s. In the last test generate, general and generous all reduce to gener, and universe and university both reduce to univers, so different words get merged (over-stemming). The opposite also happens: learners stays as learner and so does not meet the word learn, because the er rule needs a stem with m above 1 and learn only has m = 1.

In step 1c I followed the version used in NLTK, where y changes to i only after a consonant. In the original paper the word play would become plai, which would stop playing and played from sharing a stem with play in the way the question expects.

# Question 6: Implementation of N-gram Language Models

*BTL 3 – Apply | 10 Marks*

## Problem Statement

Develop a Python program that tokenizes an English paragraph, generates unigram, bigram and trigram sequences, calculates their frequencies, calculates the probability of selected bigrams and trigrams, and compares the three models.

## Aim and Concept

An N-gram language model predicts a word from the previous N-1 words. A unigram model ignores the history and only uses how common each word is. A bigram model looks at one previous word, and a trigram model at two. The probabilities are estimated from counts: P(w2 | w1) = count(w1 w2) / count(w1), and P(w3 | w1 w2) = count(w1 w2 w3) / count(w1 w2).

## Approach

1. Convert the paragraph to lowercase and pull out the words with a regular expression, which also removes punctuation.
2. Slide a window of size 1, 2 and 3 over the token list to build the unigram, bigram and trigram sequences.
3. Count them with collections.Counter and print the most frequent ones.
4. Use the count formulas to work out the probability of some chosen bigrams and trigrams.
5. Compare the three models by the total number of n-grams, the number of distinct n-grams and how many of them occur only once.

## Python Program

```
# Question 6: Unigram, bigram and trigram language models
import re
import textwrap
from collections import Counter


paragraph = ("Natural language processing helps computers understand human language. "
             "Language models predict the next word in a sentence. A good language model "
             "gives a high probability to a natural sentence. Language models are used in "
             "speech recognition and machine translation. Models learn the probability of "
             "a word from the words that come before it.")


# 1. tokenize (lowercase, keep only words)
tokens = re.findall(r"[a-z]+", paragraph.lower())
print(f"Total tokens: {len(tokens)}   Vocabulary size: {len(set(tokens))}\n")


# 2. generate n-grams
def ngrams(seq, n):
    return [tuple(seq[i:i + n]) for i in range(len(seq) - n + 1)]


uni, bi, tri = ngrams(tokens, 1), ngrams(tokens, 2), ngrams(tokens, 3)
first_bi = ", ".join(" ".join(b) for b in bi[:5])
first_tri = ", ".join(" ".join(t) for t in tri[:4])
print(textwrap.fill("First bigrams : " + first_bi, 78, subsequent_indent="   "))
print(textwrap.fill("First trigrams: " + first_tri, 78, subsequent_indent="   ") + "\n")


# 3. frequencies
uni_c, bi_c, tri_c = Counter(uni), Counter(bi), Counter(tri)


def top(counter, k=6):
    for gram, c in counter.most_common(k):
```

```
        print(f"   {' '.join(gram):<28}{c}")


print("Top unigrams");  top(uni_c)
print("Top bigrams");   top(bi_c)
print("Top trigrams");  top(tri_c)


# 4. probabilities
#    P(w2 | w1)     = count(w1 w2) / count(w1)
#    P(w3 | w1 w2)  = count(w1 w2 w3) / count(w1 w2)
def p_bigram(w1, w2):
    return bi_c[(w1, w2)] / uni_c[(w1,)]


def p_trigram(w1, w2, w3):
    return tri_c[(w1, w2, w3)] / bi_c[(w1, w2)]


print("\nBigram probabilities")
for w1, w2 in [("language", "models"), ("language", "model"), ("the", "next"),
               ("a", "natural"), ("models", "learn")]:
    ratio = f"{bi_c[(w1, w2)]}/{uni_c[(w1,)]}"
    print(f"   P({w2} | {w1})".ljust(30) + f"= {ratio} = {p_bigram(w1, w2):.3f}")


print("\nTrigram probabilities")
for t in [("language", "models", "predict"), ("language", "models", "are"),
          ("a", "high", "probability"), ("the", "probability", "of")]:
    num, den = tri_c[t], bi_c[t[:2]]
    p = num / den if den else 0
    print(f"   P({t[2]} | {t[0]} {t[1]})".ljust(36) + f"= {num}/{den} = {p:.3f}")


# 5. compare the three models
print("\nComparison of the models")
print(f"{'Model':<10}{'Total':<8}{'Distinct':<10}{'Seen once':<11}{'% seen once'}")
for name, grams, c in [("Unigram", uni, uni_c), ("Bigram", bi, bi_c), ("Trigram", tri, tri_c)]:
    once = sum(1 for v in c.values() if v == 1)
    print(f"{name:<10}{len(grams):<8}{len(c):<10}{once:<11}{100 * once / len(c):.1f}%")
```

## Output

```
Terminal  -  python question_06.py

Total tokens: 53   Vocabulary size: 36

First bigrams : natural language, language processing, processing helps, helps
   computers, computers understand
First trigrams: natural language processing, language processing helps,
   processing helps computers, helps computers understand

Top unigrams
   language                    5
   a                           5
   models                      3
   the                         3
   natural                     2
   word                        2
Top bigrams
   language models             2
   natural language            1
   language processing         1
   processing helps            1
   helps computers             1
   computers understand        1
Top trigrams
   natural language processing 1
   language processing helps   1
   processing helps computers  1
   helps computers understand  1
   computers understand human  1
   understand human language   1
```

*Output 6.1 (part 1 of 2)*

```
Terminal  -  python question_06.py

Bigram probabilities
   P(models | language)       = 2/5 = 0.400
   P(model | language)        = 1/5 = 0.200
   P(next | the)              = 1/3 = 0.333
   P(natural | a)             = 1/5 = 0.200
   P(learn | models)          = 1/3 = 0.333

Trigram probabilities
   P(predict | language models)     = 1/2 = 0.500
   P(are | language models)         = 1/2 = 0.500
   P(probability | a high)          = 1/1 = 1.000
   P(of | the probability)          = 1/1 = 1.000

Comparison of the models
Model     Total   Distinct  Seen once  % seen once
Unigram   53      36        27         75.0%
Bigram    52      51        50         98.0%
Trigram   51      51        51         100.0%
```

*Output 6.2 (part 2 of 2)*

## Analysis and Interpretation

The paragraph has 53 tokens and 36 different words. The most common words are language and a, with 5 occurrences each. The most common bigram is language models, which appears twice, and every trigram appears only once.

The probabilities show how the history helps. P(models | language) is 2/5 = 0.400, because language is followed by models in two of its five occurrences. When two words of history are given, P(predict | language models) = 0.5 and P(are | language models) = 0.5, so the model is down to two likely continuations. For a high, the trigram gives P(probability | a high) = 1.000. That looks very confident, but it comes from one single example, so it does not really mean that probability always follows a high.

The comparison table shows the main trade-off. As N goes up, the share of n-grams that appear only once rises from 75% for unigrams to 98% for bigrams and 100% for trigrams. Longer contexts make more specific predictions, but they are seen less often, so a small corpus cannot support them. This is the data sparsity problem, and it is the reason smoothing is needed (Question 7). I also did not add sentence start and end markers here, so some bigrams, like language language, are made from the end of one sentence and the start of the next. The next question handles this properly.

# Question 7: Word Frequency and N-gram Smoothing

*BTL 3 – Apply | 10 Marks*

## Problem Statement

Write a Python program that counts word frequencies in a corpus, generates bigram frequencies, calculates unsmoothed bigram probabilities, applies Add-One (Laplace) smoothing to handle unseen bigrams, and compares the probabilities before and after smoothing.

## Aim and Concept

With plain counts (maximum likelihood estimation) the probability of a bigram is count(w1 w2) / count(w1). A bigram that never occurred in the corpus gets probability 0. This is a problem because a single zero makes the probability of a whole sentence zero, even when the sentence is perfectly good English.

Add-One smoothing fixes this by pretending that every possible bigram was seen one more time: P(w2 | w1) = (count(w1 w2) + 1) / (count(w1) + V), where V is the vocabulary size. The V in the denominator keeps the probabilities of all the words after w1 adding up to 1.

## Approach

1. Split a small corpus into sentences, add <s> and </s> markers, and count each word. V is the number of distinct tokens.
2. Count every bigram.
3. Compute the unsmoothed probability of the chosen bigrams.
4. Compute the Add-One probability using the formula above.
5. Print both values for seen and unseen bigrams, check that the probabilities still add up to 1, and find how much probability goes to unseen words.

## Python Program

```
# Question 7: Word frequency and Add-One (Laplace) smoothing for bigrams
import re
import textwrap
from collections import Counter


corpus = ["the cat sat on the mat",
          "the dog sat on the rug",
          "the cat chased the dog",
          "a dog chased a cat"]


# add start and end markers so that the first and last words get a context too
sentences = [["<s>"] + re.findall(r"[a-z]+", s.lower()) + ["</s>"] for s in corpus]


# 1. word frequencies
words = [w for s in sentences for w in s]
uni = Counter(words)
V = len(uni)
print(f"Vocabulary size V = {V}")
counts = ", ".join(f"{w}: {c}" for w, c in uni.most_common())
print(textwrap.fill("Word counts: " + counts, 78, subsequent_indent="    ") + "\n")


# 2. bigram frequencies
bi = Counter((s[i], s[i + 1]) for s in sentences for i in range(len(s) - 1))
print("Bigram counts (top 8):")
for (a, b), c in bi.most_common(8):
    print(f"   ({a}, {b})".ljust(20) + str(c))
```

```
# 3. unsmoothed probability:  P(w2|w1) = c(w1,w2) / c(w1)
def p_mle(w1, w2):
    return bi[(w1, w2)] / uni[w1]


# 4. Add-one smoothing:  P(w2|w1) = (c(w1,w2) + 1) / (c(w1) + V)
def p_laplace(w1, w2):
    return (bi[(w1, w2)] + 1) / (uni[w1] + V)


# 5. compare before and after
tests = [("the", "cat"), ("the", "dog"), ("cat", "sat"), ("sat", "on"),
         ("dog", "chased"), ("the", "rug"),          # seen
         ("cat", "rug"), ("dog", "mat"), ("sat", "chased"), ("a", "mat")]  # unseen


print("\n" + "-" * 66)
print(f"{'Bigram':<18}{'Count':<8}{'Unsmoothed':<14}{'Add-one':<12}{'Change'}")
print("-" * 66)
for w1, w2 in tests:
    before, after = p_mle(w1, w2), p_laplace(w1, w2)
    pair = w1 + " " + w2
    print(f"{pair:<18}{bi[(w1, w2)]:<8}{before:<14.4f}{after:<12.4f}{after - before:+.4f}")


# sanity check: probabilities for one context must still add up to 1
vocab = sorted(uni)
for w1 in ["the", "cat"]:
    total_mle = sum(p_mle(w1, w) for w in vocab)
    total_lap = sum(p_laplace(w1, w) for w in vocab)
    print(f"\nSum of P(. | {w1}):  unsmoothed = {total_mle:.4f}   add-one = {total_lap:.4f}")


# how much probability mass goes to unseen bigrams after smoothing?
for w1 in ["the", "cat"]:
    unseen = sum(p_laplace(w1, w) for w in vocab if bi[(w1, w)] == 0)
    print(f"Mass given to unseen words after '{w1}': {unseen:.4f}")
```

## Output

```
Terminal  -  python question_07.py

Vocabulary size V = 11
Word counts: the: 6, <s>: 4, </s>: 4, cat: 3, dog: 3, sat: 2, on: 2, chased:
   2, a: 2, mat: 1, rug: 1

Bigram counts (top 8):
   (<s>, the)       3
   (the, cat)       2
   (sat, on)        2
   (on, the)        2
   (the, dog)       2
   (cat, sat)       1
   (the, mat)       1
   (mat, </s>)      1

-----------------------------------------------------------------
Bigram            Count   Unsmoothed    Add-one     Change
-----------------------------------------------------------------
the cat           2       0.3333        0.1765      -0.1569
the dog           2       0.3333        0.1765      -0.1569
cat sat           1       0.3333        0.1429      -0.1905
sat on            2       1.0000        0.2308      -0.7692
dog chased        1       0.3333        0.1429      -0.1905
the rug           1       0.1667        0.1176      -0.0490
cat rug           0       0.0000        0.0714      +0.0714
dog mat           0       0.0000        0.0714      +0.0714
sat chased        0       0.0000        0.0769      +0.0769
a mat             0       0.0000        0.0769      +0.0769
```

*Output 7.1 (part 1 of 2)*

```
Terminal  -  python question_07.py

Sum of P(. | the):  unsmoothed = 1.0000   add-one = 1.0000

Sum of P(. | cat):  unsmoothed = 1.0000   add-one = 1.0000
Mass given to unseen words after 'the': 0.4118
Mass given to unseen words after 'cat': 0.5714
```

*Output 7.2 (part 2 of 2)*

## Analysis and Interpretation

The corpus has V = 11 tokens, counting the two markers. In the table the unseen bigrams (cat rug, dog mat, sat chased and a mat) all had probability 0 before smoothing, and they get values between 0.0714 and 0.0769 afterwards, so the model can now give them some probability.

The price is paid by the bigrams that were seen. The probability of sat on went from 1.0000 to 0.2308, and the probability of the cat from 0.3333 to 0.1765. The drop is large because the corpus is tiny: sat was seen only twice, but all 11 vocabulary words receive an extra count. The sum check shows that both versions add up to exactly 1.0000, so the probability was moved around and not created.

Add-One is also quite blunt. After the word the, 41.18% of the probability goes to words that never followed it, and after cat it is 57.14%. This is much more than we would expect for unseen events.

With a small corpus the effect is strong. Real systems use smaller additions (add-k), Good-Turing or Kneser-Ney smoothing, but Laplace smoothing is still a good first step to understand the idea.

# Question 8: Entropy Calculation for Language Models

*BTL 4 – Analyze | 10 Marks*

## Problem Statement

Develop a Python program that tokenizes a corpus, calculates unigram and bigram probabilities, computes entropy with Shannon's formula, compares the unigram and bigram values, and analyses how the preceding word changes the uncertainty of the next word.

## Aim and Concept

Entropy measures uncertainty. Shannon's formula is H = - sum of p(x) log2 p(x), and the answer is in bits. If a model is very sure of what comes next, the entropy is low, and if many words are about equally likely, it is high. Perplexity, 2 to the power H, says the same thing as the number of equally likely choices the model is torn between.

For the unigram model I use p(w) = count(w) / N. For the bigram model the useful quantity is the conditional entropy H(W2 | W1) = - sum of p(w1, w2) log2 p(w2 | w1), which is the average uncertainty about the next word when the previous word is known.

## Approach

1. Tokenize the corpus into lowercase words.
2. Count unigrams and bigrams and turn the counts into probabilities (joint and conditional).
3. Compute the unigram entropy, the joint bigram entropy and the conditional entropy using log base 2.
4. Compare the values and compute the perplexity of each model.
5. For some chosen previous words, calculate the entropy of the words that follow them.

## Python Program

```
# Question 8: Entropy of a corpus with unigram and bigram models
import re
import math
from collections import Counter, defaultdict


corpus = ("""the students opened the book and the teacher explained the lesson to the class.
the teacher asked a question and the students answered the question with care.
a good student reads the book before the class and asks the teacher about the lesson.
the class listened to the teacher and the students wrote the answer in the book.
the teacher gave the students a test and the students finished the test before the bell.
a student who reads the lesson twice can answer the question in the test.""")


# 1. tokenize
tokens = re.findall(r"[a-z]+", corpus.lower())
N = len(tokens)
print(f"Tokens: {N}    Distinct words: {len(set(tokens))}\n")


# 2. unigram probabilities  P(w) = c(w) / N
uni = Counter(tokens)
p_uni = {w: c / N for w, c in uni.items()}


# bigram probabilities  P(w1,w2) = c(w1,w2) / (N-1)  and  P(w2|w1) = c(w1,w2) / c(w1 as first)
pairs = list(zip(tokens, tokens[1:]))
bi = Counter(pairs)
first = Counter(w1 for w1, _ in pairs)
p_joint = {b: c / len(pairs) for b, c in bi.items()}
p_cond = {(w1, w2): c / first[w1] for (w1, w2), c in bi.items()}
```

```
# 3. Shannon entropy  H = - sum p * log2(p)
H_uni = -sum(p * math.log2(p) for p in p_uni.values())
H_joint = -sum(p * math.log2(p) for p in p_joint.values())
# conditional entropy: uncertainty of the next word when the previous word is known
H_cond = -sum(p_joint[b] * math.log2(p_cond[b]) for b in bi)


print("Entropy results (bits)")
print("-" * 52)
print(f"Unigram entropy          H(W)      = {H_uni:.4f}   perplexity {2 ** H_uni:.2f}")
print(f"Bigram joint entropy     H(W1,W2)  = {H_joint:.4f}")
print(f"Bigram conditional       H(W2|W1)  = {H_cond:.4f}   perplexity {2 ** H_cond:.2f}")
print(f"Reduction in uncertainty   = {H_uni - H_cond:.4f} bits "
      f"({100 * (H_uni - H_cond) / H_uni:.1f}%)")


# 5. how the previous word changes the uncertainty of the next word
print("\nNext-word entropy after some chosen words")
print(f"{'Previous word':<16}{'Times seen':<12}{'Different next words':<22}{'Entropy'}")
by_prev = defaultdict(list)
for (w1, w2), c in bi.items():
    by_prev[w1].append(c)
for w1 in ["the", "students", "teacher", "a", "book", "question"]:
    counts = by_prev[w1]
    total = sum(counts)
    h = -sum((c / total) * math.log2(c / total) for c in counts)
    print(f"{w1:<16}{total:<12}{len(counts):<22}{h:.3f}")


print(f"\nWith no context at all, the maximum possible entropy for {len(uni)} words "
      f"is {math.log2(len(uni)):.3f} bits.")
```

## Output

```
Terminal  -  python question_08.py

Tokens: 88   Distinct words: 33

Entropy results (bits)
----------------------------------------------------
Unigram entropy          H(W)       = 4.2469   perplexity 18.99
Bigram joint entropy     H(W1,W2)   = 5.7822
Bigram conditional       H(W2|W1)   = 1.5455   perplexity 2.92
Reduction in uncertainty    = 2.7014 bits (63.6%)

Next-word entropy after some chosen words
Previous word   Times seen  Different next words  Entropy
the             25          9                     2.984
students        5           5                     2.322
teacher         5           5                     2.322
a               4           4                     2.000
book            3           3                     1.585
question        3           3                     1.585

With no context at all, the maximum possible entropy for 33 words is 5.044 bits.
```

*Output 8.1*

## Analysis and Interpretation

The corpus has 88 tokens and 33 different words. With no context the entropy is 4.2469 bits, which is a perplexity of about 19 and fairly close to the maximum of 5.044 bits for 33 words. When the previous word is known the conditional entropy drops to 1.5455 bits, a perplexity of 2.92. Knowing the preceding word removes 2.7014 bits, which is about 63.6% of the uncertainty.

The table of previous words shows why. After the, the next word can be one of 9 different words and the entropy is 2.984 bits, the highest in the table, because the is followed by many different nouns.

After book or question the model saw only 3 different continuations, giving 1.585 bits. In general, a context that allows only a few next words makes the model more certain. The joint entropy H(W1, W2) = 5.7822 bits is also close to the unigram entropy plus the conditional entropy (4.2469 + 1.5455 = 5.7924), which matches the chain rule.

One point needs care. The corpus is tiny, so most bigrams were seen only once and their conditional probability looks like 1. That makes the bigram entropy lower than it would be on a large corpus. The direction of the result (bigram entropy below unigram entropy) can be trusted, but the size of the drop is probably too optimistic.

# Question 9: English Part-of-Speech Tagging

*BTL 3 – Apply | 10 Marks*

## Problem Statement

Develop a Python program that tokenizes an English paragraph, tags each word with a standard English POS tagset, identifies nouns, verbs, adjectives, adverbs and prepositions, displays each word with its tag, and counts the frequency of each POS category.

## Aim and Concept

Part-of-speech (POS) tagging gives every word in a sentence a label such as noun, verb or adjective. I used the Penn Treebank tagset, the same one NLTK uses: NN and NNS for singular and plural nouns, NNP for proper nouns, VBZ, VBD, VBG and VBP for verb forms, JJ for adjectives, RB for adverbs, IN for prepositions, DT for determiners, PRP for pronouns, CC for conjunctions and a full stop for punctuation.

The tagger I wrote uses a word list plus rules. It first looks the word up in the list, and if the word is not there it falls back on clues such as a capital letter inside a sentence or the suffix (-ly adverb, -ing verb, -ed past tense, -s plural noun).

## Approach

1. Split the paragraph into sentences, and the sentences into words and punctuation marks.
2. Tag each token: punctuation and numbers first, then the word list, then the capital letter and suffix rules, and NN as the default.
3. Print each word with its tag and the meaning of the tag.
4. Map the tags to the five categories asked for: noun (NN, NNS, NNP), verb (VB*), adjective (JJ), adverb (RB) and preposition (IN).
5. Count and display the words in each category and the frequency of each tag.

## Python Program

```
# Question 9: English part-of-speech tagging with the Penn Treebank tagset
import re
import textwrap
from collections import Counter


paragraph = ("The quick brown fox jumps over the lazy dog near the river. "
             "Maria quickly reads interesting books in the old library every evening. "
             "She carefully opened the heavy door and walked slowly into the quiet room. "
             "Students are learning new skills with great enthusiasm.")


# closed-class words and a few common open-class words (Penn Treebank tags)
LEXICON = {
    "the": "DT", "a": "DT", "an": "DT", "every": "DT", "this": "DT",
    "over": "IN", "near": "IN", "in": "IN", "into": "IN", "with": "IN", "on": "IN",
    "and": "CC", "but": "CC", "or": "CC",
    "she": "PRP", "he": "PRP", "they": "PRP", "it": "PRP",
    "are": "VBP", "is": "VBZ", "was": "VBD", "were": "VBD",
    "fox": "NN", "dog": "NN", "river": "NN", "library": "NN", "door": "NN",
    "room": "NN", "evening": "NN", "enthusiasm": "NN",
    "maria": "NNP", "interesting": "JJ", "quick": "JJ", "brown": "JJ", "lazy": "JJ",
    "old": "JJ", "heavy": "JJ",
    "quiet": "JJ", "new": "JJ", "great": "JJ",
    "jumps": "VBZ", "reads": "VBZ", "books": "NNS", "skills": "NNS",
}
PREPOSITIONS = {"over", "near", "in", "into", "with", "on"}
```

```
def tag_word(word, position):
    low = word.lower()
    if re.fullmatch(r"[.,;:!?]", word):
        return "."
    if re.fullmatch(r"\d+(\.\d+)?", word):
        return "CD"
    if low in LEXICON:
        return LEXICON[low]
    if word[0].isupper() and position > 0:          # capital inside a sentence
        return "NNP"
    if low.endswith("ly"):
        return "RB"
    if low.endswith("ing"):
        return "VBG"
    if low.endswith("ed"):
        return "VBD"
    if low.endswith(("ous", "ful", "ive", "able", "al")):
        return "JJ"
    if low.endswith("s"):
        return "NNS"
    return "NN"


# 1. tokenize sentence by sentence
sentences = [s for s in re.split(r"(?<=[.!?])\s+", paragraph) if s]
tagged = []
for sent in sentences:
    toks = re.findall(r"\w+|[.,;:!?]", sent)
    tagged += [(w, tag_word(w, i)) for i, w in enumerate(toks)]


# 4. word + tag
MEANING = {"DT": "determiner", "IN": "preposition", "CC": "conjunction",
           "PRP": "pronoun", "NN": "noun (sing.)", "NNS": "noun (plural)",
           "NNP": "proper noun", "VBZ": "verb (3rd sg.)", "VBP": "verb (non-3rd)",
           "VBD": "verb (past)", "VBG": "verb (-ing)", "JJ": "adjective",
           "RB": "adverb", ".": "punctuation", "CD": "number"}


rows = [f"{w:<13}{t:<5}{MEANING.get(t, ''):<15}" for w, t in tagged]
half = (len(rows) + 1) // 2
head = f"{'Word':<13}{'Tag':<5}{'Meaning':<15}"
print(head + "| " + head)
print("-" * 36 + "+-" + "-" * 36)
for i in range(half):
    right = rows[i + half] if i + half < len(rows) else ""
    print(rows[i] + "| " + right)


# 2 & 5. group into the required categories and count them
def category(word, tag):
    if tag.startswith("NN"):  return "Noun"
    if tag.startswith("VB"):  return "Verb"
    if tag.startswith("JJ"):  return "Adjective"
    if tag.startswith("RB"):  return "Adverb"
    if tag == "IN" and word.lower() in PREPOSITIONS:  return "Preposition"
    return "Other"


cats = {}
for w, t in tagged:
    cats.setdefault(category(w, t), []).append(w)


print("\nWords by category")
for c in ["Noun", "Verb", "Adjective", "Adverb", "Preposition"]:
    line = f"{c} ({len(cats.get(c, []))}): " + ", ".join(cats.get(c, []))
    print(textwrap.fill(line, 78, initial_indent="   ", subsequent_indent="       "))


print("\nTag frequency")
for tag, n in Counter(t for _, t in tagged).most_common():
    print(f"   {tag:<5}{n:>3}  {'#' * n}")
```

## Output

Terminal - python question_09.py

```
Word         Tag  Meaning        | Word         Tag  Meaning
-----------------------------------+-----------------------------------
The          DT   determiner     | .            .    punctuation
quick        JJ   adjective      | She          PRP  pronoun
brown        JJ   adjective      | carefully    RB   adverb
fox          NN   noun (sing.)   | opened       VBD  verb (past)
jumps        VBZ  verb (3rd sg.) | the          DT   determiner
over         IN   preposition    | heavy        JJ   adjective
the          DT   determiner     | door         NN   noun (sing.)
lazy         JJ   adjective      | and          CC   conjunction
dog          NN   noun (sing.)   | walked       VBD  verb (past)
near         IN   preposition    | slowly       RB   adverb
the          DT   determiner     | into         IN   preposition
river        NN   noun (sing.)   | the          DT   determiner
.            .    punctuation    | quiet        JJ   adjective
Maria        NNP  proper noun    | room         NN   noun (sing.)
quickly      RB   adverb         | .            .    punctuation
reads        VBZ  verb (3rd sg.) | Students     NNS  noun (plural)
interesting  JJ   adjective      | are          VBP  verb (non-3rd)
books        NNS  noun (plural)  | learning     VBG  verb (-ing)
in           IN   preposition    | new          JJ   adjective
the          DT   determiner     | skills       NNS  noun (plural)
old          JJ   adjective      | with         IN   preposition
library      NN   noun (sing.)   | great        JJ   adjective
every        DT   determiner     | enthusiasm   NN   noun (sing.)
evening      NN   noun (sing.)   | .            .    punctuation
```

*Output 9.1 (part 1 of 2)*

Terminal - python question_09.py

```
Words by category
   Noun (12): fox, dog, river, Maria, books, library, evening, door, room,
       Students, skills, enthusiasm
   Verb (6): jumps, reads, opened, walked, are, learning
   Adjective (9): quick, brown, lazy, interesting, old, heavy, quiet, new,
       great
   Adverb (3): quickly, carefully, slowly
   Preposition (5): over, near, in, into, with

Tag frequency
   JJ     9  #########
   NN     8  ########
   DT     7  #######
   IN     5  #####
   .      4  ####
   RB     3  ###
   NNS    3  ###
   VBZ    2  ##
   VBD    2  ##
   NNP    1  #
   PRP    1  #
   CC     1  #
   VBP    1  #
   VBG    1  #
```

*Output 9.2 (part 2 of 2)*

## Analysis and Interpretation

The paragraph has 48 tokens. The category counts are 12 nouns, 6 verbs, 9 adjectives, 3 adverbs and 5 prepositions. The remaining tokens are determiners (7), punctuation (4), one pronoun and one conjunction. Adjectives are almost as frequent as nouns because the paragraph is descriptive (quick, brown, lazy, old, heavy, quiet and so on).

Most of the output is correct, but a few words show where a simple tagger has trouble. Interesting ends in -ing, so the suffix rule alone would call it a verb (VBG), but in interesting books it is an adjective, so I had to put it in the word list. Evening is similar because it can be a noun or the -ing form of a verb. Maria was also added to the list, since a capital letter at the start of a sentence cannot be told apart from an ordinary sentence-initial word. A statistical tagger decides such cases from the surrounding words, and this is what Question 10 compares.

The same kind of table can be produced with NLTK's pos_tag, which uses a model trained on a large tagged corpus. I wrote my own version so that every decision is visible in the code, and the tags in the output follow the same Penn Treebank scheme.

# Question 10: Comparative Analysis of POS Tagging Techniques

*BTL 4 – Analyze | 10 Marks*

## Problem Statement

Develop a Python application that implements a rule-based, a stochastic and a transformation-based POS tagger, runs them on the same sentences, compares the results, and analyses the differences in accuracy and the cases where the methods give different tags.

## Aim and Concept

There are three common ways to build a POS tagger. A rule-based tagger uses hand-written knowledge such as word lists, suffix patterns and context rules like “a word after to is a verb”. A stochastic tagger learns probabilities from a tagged corpus. I used a bigram Hidden Markov Model (HMM), which combines how likely a word is for a tag, P(word | tag), with how likely a tag is after the previous tag, P(tag | previous tag), and finds the best tag sequence with the Viterbi algorithm.

A transformation-based (Brill) tagger starts with a simple tagging and then learns an ordered list of correction rules from the training data. Each rule says “change tag A to tag B when the previous tag (or next tag, previous word, next word) is X”.

## Approach

1. Create a small hand-tagged training corpus (43 sentences, 273 tokens) and a separate test set of 8 sentences (56 tokens) with gold tags. The test set has ambiguous words such as book, play, run, fish and light, and some words that are not in the training data.
2. Rule-based tagger: a word list for closed-class words, then suffix rules, then context rules.
3. HMM tagger: count transitions and emissions, smooth the transitions with add-0.1, use the last two letters of an unseen word for its emission probability, and decode with Viterbi.
4. Brill tagger: start from the most frequent tag of each training word (a suffix guess for new words). Then repeatedly pick the transformation that fixes the most mistakes and keep it only if the net gain is at least 2.
5. Tag the test sentences with all three taggers, compare with the gold tags, and print the accuracy and the tokens where the taggers disagree.

## Python Program

```
# Question 10: Rule-based vs stochastic (HMM) vs transformation-based (Brill) POS tagging
import math
from collections import Counter, defaultdict


# ---------------------------------------------------------------- data
def parse(block):
    return [[tuple(tok.rsplit("/", 1)) for tok in line.split()]
            for line in block.strip().splitlines()]


TRAIN = parse("""
The/DT dog/NN runs/VBZ in/IN the/DT park/NN ./.
I/PRP read/VBD a/DT book/NN yesterday/NN ./.
She/PRP wants/VBZ to/TO book/VB a/DT flight/NN ./.
They/PRP can/MD play/VB football/NN ./.
The/DT children/NNS watch/VBP a/DT play/NN ./.
He/PRP likes/VBZ to/TO run/VB in/IN the/DT morning/NN ./.
The/DT run/NN was/VBD long/JJ ./.
```

```
We/PRP will/MD walk/VB to/TO school/NN ./.
A/DT quick/JJ fox/NN jumps/VBZ over/IN a/DT lazy/JJ dog/NN ./.
The/DT teacher/NN reads/VBZ the/DT book/NN to/TO the/DT class/NN ./.
Students/NNS study/VBP hard/RB for/IN the/DT test/NN ./.
The/DT light/NN is/VBZ bright/JJ ./.
This/DT bag/NN is/VBZ light/JJ ./.
Birds/NNS can/MD fly/VB high/RB ./.
The/DT fish/NN swims/VBZ in/IN the/DT river/NN ./.
We/PRP fish/VBP in/IN the/DT lake/NN ./.
She/PRP can/MD drive/VB a/DT car/NN ./.
He/PRP put/VBD the/DT fish/NN in/IN a/DT can/NN ./.
The/DT old/JJ man/NN walks/VBZ slowly/RB ./.
Maria/NNP plays/VBZ the/DT piano/NN well/RB ./.
They/PRP watched/VBD the/DT game/NN on/IN television/NN ./.
I/PRP like/VBP to/TO watch/VB movies/NNS ./.
The/DT new/JJ plant/NN grows/VBZ fast/RB ./.
We/PRP plant/VBP trees/NNS every/DT year/NN ./.
The/DT big/JJ dog/NN barked/VBD loudly/RB ./.
She/PRP is/VBZ reading/VBG a/DT good/JJ book/NN ./.
Rahul/NNP bought/VBD a/DT new/JJ phone/NN ./.
The/DT cat/NN sat/VBD on/IN the/DT mat/NN ./.
They/PRP will/MD book/VB the/DT tickets/NNS ./.
He/PRP wrote/VBD a/DT short/JJ story/NN ./.
The/DT story/NN was/VBD very/RB interesting/JJ ./.
Many/JJ students/NNS enjoy/VBP the/DT play/NN ./.
Kids/NNS play/VBP in/IN the/DT garden/NN ./.
I/PRP need/VBP a/DT drink/NN ./.
We/PRP drink/VBP water/NN daily/RB ./.
The/DT teacher/NN gave/VBD the/DT students/NNS a/DT test/NN ./.
The/DT workers/NNS run/VBP the/DT machine/NN ./.
The/DT bright/JJ sun/NN rises/VBZ early/RB ./.
She/PRP was/VBD happy/JJ with/IN the/DT result/NN ./.
The/DT boys/NNS and/CC the/DT girls/NNS play/VBP together/RB ./.
He/PRP slowly/RB opened/VBD the/DT door/NN ./.
They/PRP quietly/RB left/VBD the/DT room/NN ./.
She/PRP often/RB walks/VBZ to/TO school/NN ./.
""")


TEST = parse("""
The/DT children/NNS play/VBP in/IN the/DT garden/NN ./.
I/PRP can/MD book/VB a/DT flight/NN ./.
The/DT book/NN is/VBZ very/RB light/JJ ./.
They/PRP watch/VBP the/DT play/NN ./.
She/PRP quickly/RB ran/VBD to/TO the/DT market/NN ./.
The/DT lazy/JJ cat/NN sleeps/VBZ on/IN the/DT soft/JJ bed/NN ./.
We/PRP run/VBP fast/RB and/CC the/DT run/NN ends/VBZ soon/RB ./.
He/PRP will/MD fish/VB near/IN the/DT river/NN ./.
""")


# ---------------------------------------------------- 1. rule-based tagger
CLOSED = {"the": "DT", "a": "DT", "an": "DT", "this": "DT", "every": "DT",
          "in": "IN", "on": "IN", "over": "IN", "near": "IN", "with": "IN", "for": "IN",
          "and": "CC", "but": "CC", "or": "CC", "to": "TO",
          "i": "PRP", "he": "PRP", "she": "PRP", "we": "PRP", "they": "PRP",
          "can": "MD", "will": "MD", "would": "MD", "could": "MD",
          "is": "VBZ", "are": "VBP", "was": "VBD", "were": "VBD",
          "very": "RB", "fast": "RB", "not": "RB",
          "lazy": "JJ", "quick": "JJ", "big": "JJ", "old": "JJ",
          "new": "JJ", "good": "JJ", "long": "JJ", "bright": "JJ", "happy": "JJ"}


def rule_tagger(words):
    tags = []
    for i, w in enumerate(words):
        low = w.lower()
        prev = tags[-1] if tags else "START"
        if w in ".,;:!?":
            tag = "."                           # punctuation
        elif low in CLOSED:
            tag = CLOSED[low]                       # word list lookup
        elif w[0].isupper() and i > 0:
            tag = "NNP"                          # capital inside a sentence
        elif low.endswith("ly"):
            tag = "RB"                          # suffix rules
        elif low.endswith("ing"):
            tag = "VBG"
        elif low.endswith("ed"):
            tag = "VBD"
        elif prev in ("MD", "TO"):
            tag = "VB"                          # context rules start here
        elif prev == "RB":
            tag = "JJ"
        elif prev == "PRP":
            tag = "VBP"
        elif prev in ("DT", "JJ"):
            tag = "NNS" if low.endswith("s") else "NN"
```

```
        elif prev in ("NN", "NNS", "NNP"):
            tag = "VBZ" if low.endswith("s") else "VBP"
        else:
            tag = "NN"
        tags.append(tag)
    return tags


# ---------------------------------------------------- 2. stochastic (HMM + Viterbi)
class HMMTagger:
    def __init__(self, data):
        self.trans, self.emit = defaultdict(Counter), defaultdict(Counter)
        self.tag_n, self.suf = Counter(), defaultdict(Counter)
        for sent in data:
            prev = "<s>"
            for w, t in sent:
                self.trans[prev][t] += 1
                self.emit[t][w.lower()] += 1
                self.suf[t][w.lower()[-2:]] += 1
                self.tag_n[t] += 1
                prev = t
        self.tags = list(self.tag_n)
        self.vocab = {w for t in self.emit for w in self.emit[t]}


    def p_trans(self, a, b):                            # add-0.1 smoothing
        return (self.trans[a][b] + 0.1) / (sum(self.trans[a].values()) + 0.1 * len(self.tags))


    def p_emit(self, w, t):
        w = w.lower()
        if w in self.vocab:
            return (self.emit[t][w] + 0.001) / (self.tag_n[t] + 0.001 * len(self.vocab))
        return (self.suf[t][w[-2:]] + 0.5) / (self.tag_n[t] + 5)   # unknown word: use its ending


    def tag(self, words):
        best = [{t: (math.log(self.p_trans("<s>", t)) + math.log(self.p_emit(words[0], t)), None)
                 for t in self.tags}]
        for w in words[1:]:
            col = {}
            for t in self.tags:
                score, back = max((best[-1][p][0] + math.log(self.p_trans(p, t)), p)
                                  for p in self.tags)
                col[t] = (score + math.log(self.p_emit(w, t)), back)
            best.append(col)
        t = max(best[-1], key=lambda k: best[-1][k][0])
        out = [t]
        for col in reversed(best[1:]):
            t = col[t][1]
            out.append(t)
        return out[::-1]


# ---------------------------------------------------- 3. transformation-based (Brill)
def guess_tag(word, position):
    """Backoff for unknown words: simple suffix patterns."""
    low = word.lower()
    if word[0].isupper() and position > 0: return "NNP"
    if low.endswith("ly"):                 return "RB"
    if low.endswith("ing"):                return "VBG"
    if low.endswith("ed"):                 return "VBD"
    if low.endswith("s"):                  return "NNS"
    return "NN"


class BrillTagger:
    def __init__(self, data, max_rules=20):
        counts = defaultdict(Counter)
        for sent in data:
            for w, t in sent:
                counts[w.lower()][t] += 1
        # initial tagger: most frequent tag of each training word, suffix guess otherwise
        self.best = {w: c.most_common(1)[0][0] for w, c in counts.items()}
        self.rules = self.learn(data, max_rules)


    def initial(self, words):
        return [self.best.get(w.lower(), guess_tag(w, i)) for i, w in enumerate(words)]


    @staticmethod
    def conditions(words, tags, i):
        prev_t = tags[i - 1] if i else "START"
        next_t = tags[i + 1] if i + 1 < len(tags) else "END"
        prev_w = words[i - 1].lower() if i else "START"
        next_w = words[i + 1].lower() if i + 1 < len(words) else "END"
        conds = [("prev tag", prev_t), ("next tag", next_t),
                 ("prev word", prev_w), ("next word", next_w)]
        return conds
```

```
    def learn(self, data, max_rules):
        words = [[w for w, _ in s] for s in data]
        gold = [[t for _, t in s] for s in data]
        cur = [self.initial(w) for w in words]
        rules = []
        for _ in range(max_rules):
            # conditions for every token under the current tagging
            conds = [[self.conditions(ws, cs, i) for i in range(len(ws))]
                     for ws, cs in zip(words, cur)]
            # candidate rules come from the mistakes that are still left
            cands = {(cs[i], gs[i], c)
                     for ws, gs, cs, cn in zip(words, gold, cur, conds)
                     for i in range(len(ws)) if gs[i] != cs[i] for c in cn[i]}
            best, best_score = None, 0
            for rule in sorted(cands, key=str):
                frm, to, cond = rule
                score = 0                   # fixes minus breaks
                for gs, cs, cn in zip(gold, cur, conds):
                    for i in range(len(gs)):
                        if cs[i] == frm and cond in cn[i]:
                            score += (gs[i] == to) - (gs[i] == frm)
                if score > best_score:
                    best, best_score = rule, score
            if best is None or best_score < 2:      # skip rules that fix only one token
                break
            rules.append(best)
            for ws, cs in zip(words, cur):
                self.apply(best, ws, cs)
        return rules


    @classmethod
    def apply(cls, rule, words, tags):
        frm, to, cond = rule
        snapshot = list(tags)
        for i in range(len(words)):
            if snapshot[i] == frm and cond in cls.conditions(words, snapshot, i):
                tags[i] = to


    def tag(self, words):
        tags = self.initial(words)
        for rule in self.rules:
            self.apply(rule, words, tags)
        return tags


# ---------------------------------------------------------------- run everything
hmm, brill = HMMTagger(TRAIN), BrillTagger(TRAIN)
print(f"Training: {len(TRAIN)} sentences, {sum(map(len, TRAIN))} tokens   "
      f"Test: {len(TEST)} sentences, {sum(map(len, TEST))} tokens\n")


print("Transformation rules learned by the Brill tagger")
for n, (frm, to, (kind, val)) in enumerate(brill.rules, 1):
    print(f"  {n:>2}. change {frm} -> {to} when {kind}: '{val}'")


taggers = {"Rule": rule_tagger, "HMM": hmm.tag, "Brill": brill.tag}
correct, total = Counter(), 0
diff_rows = []
print("\nTagging results (* marks a wrong tag)")
for s_no, sent in enumerate(TEST, 1):
    words, gold = [w for w, _ in sent], [t for _, t in sent]
    out = {name: f(words) for name, f in taggers.items()}
    print(f"\nSentence {s_no}: {' '.join(words)}")
    print(f"  {'Word':<10}{'Gold':<6}{'Rule':<7}{'HMM':<7}{'Brill':<7}")
    for i, w in enumerate(words):
        cells = []
        for name in taggers:
            ok = out[name][i] == gold[i]
            correct[name] += ok
            cells.append(out[name][i] + ("" if ok else "*"))
        if len({out[n][i] for n in taggers}) > 1:
            diff_rows.append((w, gold[i], *[out[n][i] for n in taggers]))
        print(f"  {w:<10}{gold[i]:<6}{cells[0]:<7}{cells[1]:<7}{cells[2]:<7}")
    total += len(words)


print("\nAccuracy on the test sentences")
for name in taggers:
    print(f"  {name:<6}{correct[name]:>3}/{total}  = {100 * correct[name] / total:.1f}%")


print(f"\nTokens where the taggers disagree: {len(diff_rows)}")
print(f"  {'Word':<10}{'Gold':<6}{'Rule':<6}{'HMM':<6}{'Brill'}")
for row in diff_rows:
    print(f"  {row[0]:<10}{row[1]:<6}{row[2]:<6}{row[3]:<6}{row[4]}")
```

## Output

```
Terminal  -  python question_10.py

Training: 43 sentences, 273 tokens   Test: 8 sentences, 56 tokens

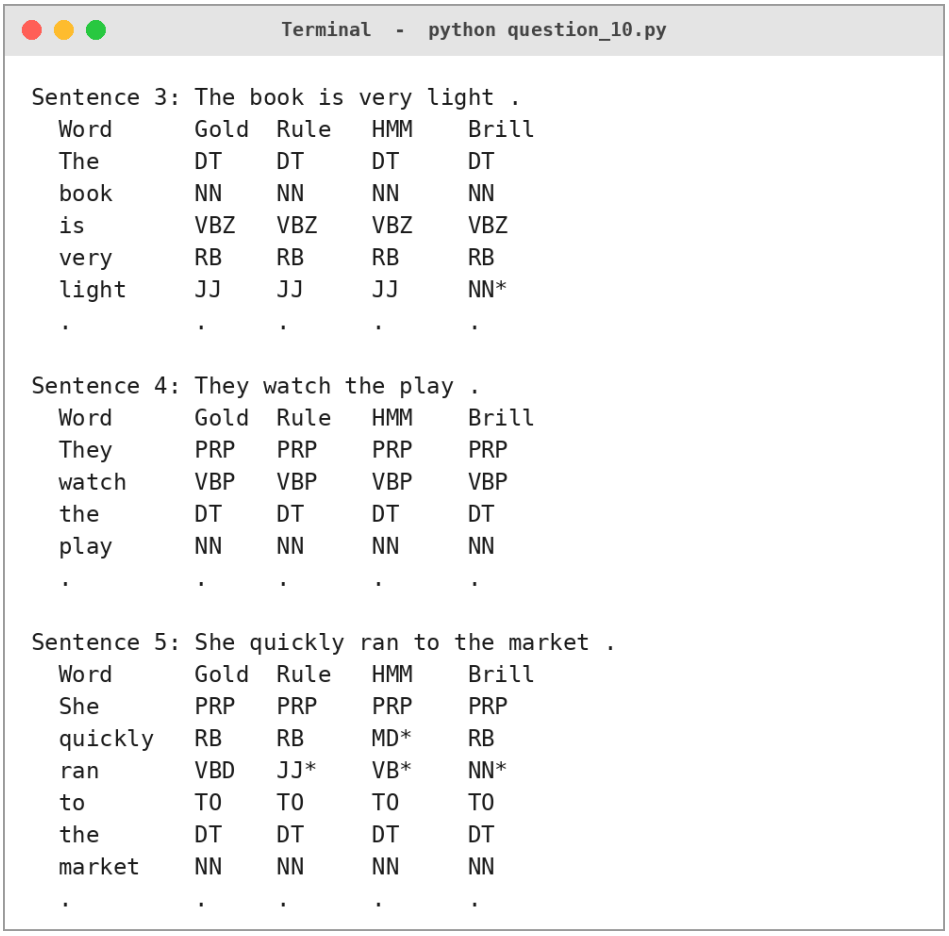
Transformation rules learned by the Brill tagger
   1. change NN -> VBP when prev tag: 'PRP'
   2. change NN -> VB when next tag: 'DT'
   3. change NN -> VBP when prev tag: 'NNS'

Tagging results (* marks a wrong tag)

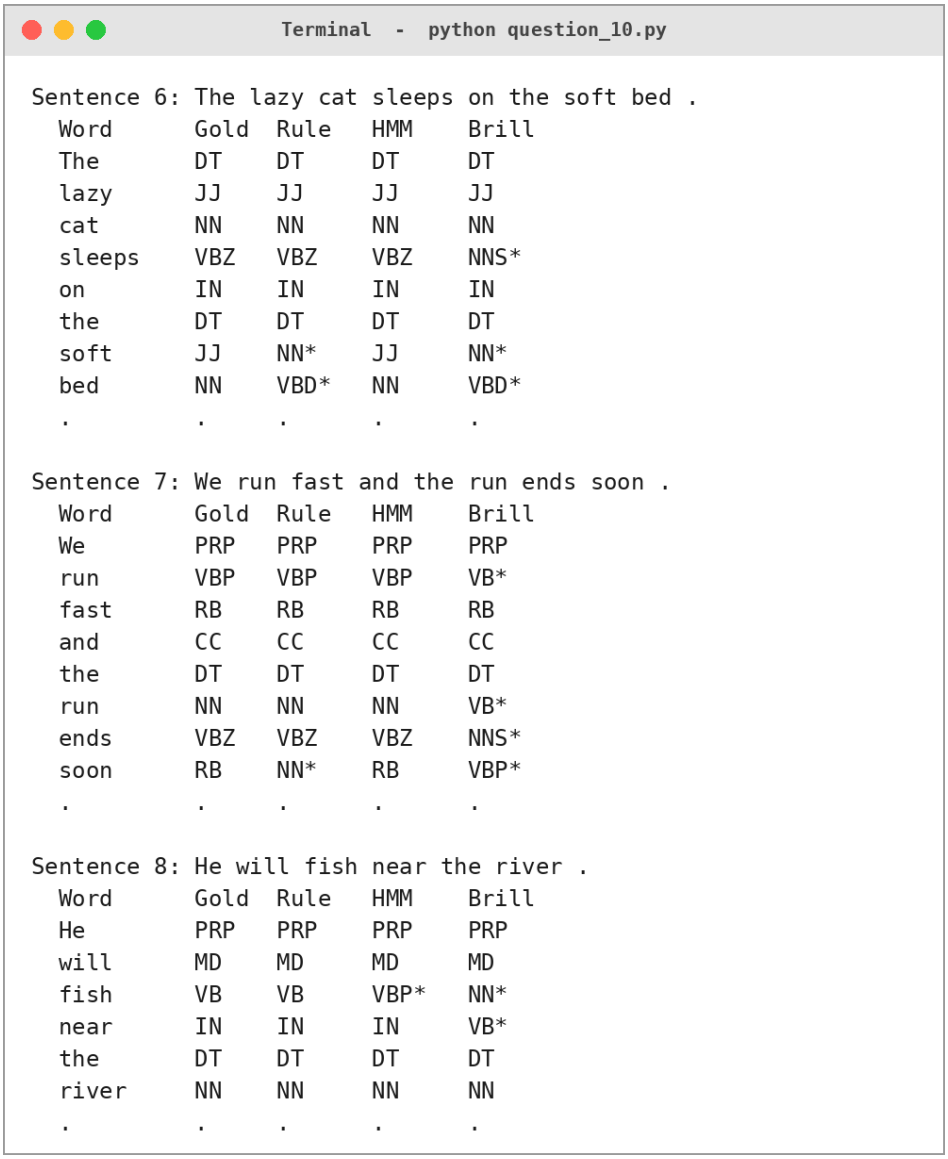
Sentence 1: The children play in the garden .
  Word      Gold  Rule   HMM    Brill
  The       DT    DT     DT     DT
  children  NNS   NN*    NNS    NNS
  play      VBP   VBP    VBP    VBP
  in        IN    IN     IN     IN
  the       DT    DT     DT     DT
  garden    NN    NN     NN     NN
  .         .     .      .      .

Sentence 2: I can book a flight .
  Word      Gold  Rule   HMM    Brill
  I         PRP   PRP    PRP    PRP
  can       MD    MD     MD     MD
  book      VB    VB     VB     VB
  a         DT    DT     DT     DT
  flight    NN    NN     NN     NN
  .         .     .      .      .
```

*Output 10.1 (part 1 of 4)*




*Output 10.2 (part 2 of 4)*




*Output 10.3 (part 3 of 4)*

```
Terminal  -  python question_10.py

Accuracy on the test sentences
  Rule   51/56  = 91.1%
  HMM    53/56  = 94.6%
  Brill  45/56  = 80.4%

Tokens where the taggers disagree: 13
  Word      Gold  Rule  HMM   Brill
  children  NNS   NN    NNS   NNS
  light     JJ    JJ    JJ    NN
  quickly   RB    RB    MD    RB
  ran       VBD   JJ    VB    NN
  sleeps    VBZ   VBZ   VBZ   NNS
  soft      JJ    NN    JJ    NN
  bed       NN    VBD   NN    VBD
  run       VBP   VBP   VBP   VB
  run       NN    NN    NN    VB
  ends      VBZ   VBZ   VBZ   NNS
  soon      RB    NN    RB    VBP
  fish      VB    VB    VBP   NN
  near      IN    IN    IN    VB
```

*Output 10.4 (part 4 of 4)*

## Analysis and Interpretation

On the 56 test tokens the HMM scored 94.6% (53 correct), the rule-based tagger 91.1% (51) and the Brill tagger 80.4% (45). The taggers disagree on 13 tokens, and these show the strengths and weaknesses of each approach. A star in the output marks a wrong tag.

The rule-based tagger is strong where its rules apply: closed-class words, the -ly suffix and words after to or a modal. Its mistakes come from words its rules do not cover. It tagged children as NN, because its only knowledge is that a word after a determiner that does not end in s is a singular noun. It tagged soft as NN since soft is not on its word list, bed as VBD since the -ed rule fires on spelling, and soon as NN.

The HMM did best because it uses the word and its neighbours together. It got children, soft, bed and soon right. Its three errors are quickly (tagged MD), ran (VB) and fish (VBP). Quickly was not in the training data, and in several training sentences a pronoun is followed by a modal, so the transition probabilities pushed the tag to MD even though the -ly ending points to an adverb. That wrong tag then affected the next word, ran, which shows that an HMM can carry one mistake forward. Fish appeared in the training data only as a noun and as a plain present verb, never as VB after will, so the model had no direct evidence for that reading.

The Brill tagger was the weakest here. With only 43 training sentences it learned just 3 rules: NN to VBP after a pronoun, NN to VB before a determiner, and NN to VBP after a plural noun. Most of its errors are unknown words that only got a suffix guess such as NN, NNS or VBD. Ends was guessed as NNS, and then soon, which follows it, was changed to VBP by the third rule. The rule “NN to VB before a determiner” helps verbs that are followed by an object, but it also changed near to VB. Brill’s method normally uses thousands of sentences, so this result says more about the small training set than about the method.

Some words are hard for every tagger. Ran is wrong in all three outputs because it is an irregular past tense with no suffix to give it away and it was not in the training data. The ambiguous words book and play were correct in all three taggers, but run and light, which have several tags, were mixed up by the Brill tagger, because the most frequent tag in its list does not suit every sentence and none of its three rules corrected it.

The conclusions are limited by the small test set. With 56 tokens each token is worth about 1.8%, and the gap between the HMM and the rule-based tagger is only two tokens. What the experiment does show is the kind of mistake each method makes. Rules fail on words outside the rules, the HMM fails when an unseen word and a strong tag transition disagree, and the Brill tagger fails when it has too little data to learn useful corrections. In practice the methods are often combined, for example by starting a Brill tagger from a stronger initial tagger.